

Switch Case

Content notes, classwork and homework with worked C++ solutions.

Lesson 1: The switch Statement

Quick recap: switch statement • case and break • default; fall-through

Theory & Nuances

switch statement

An alternative to a long else-if ladder, used when one variable is compared against many fixed constant values (numbers or single characters). It reads top-to-bottom as "jump straight to the matching case" rather than testing conditions one at a time.

case and break

Each case label marks an entry point into the switch. `break` exits the switch immediately after that case's code runs. Without `break`, execution falls through into the next case's code regardless of whether its label matches — this is a deliberate C++ feature, not an error, but an easy trap if forgotten.

default and fall-through

`default` is the fallback branch that runs when no case label matches — it plays the same role as the final `else` in an if-else ladder. Intentional fall-through (stacking several case labels with no `break` between them to share one block of code) is a legitimate pattern, but should be obvious/commented so it doesn't look like a missing `break`.

Key Points / Common Mistakes

- `switch` only tests exact equality against constant values — no ranges, no relational operators.
- A missing `break` is one of the most common switch bugs — code quietly falls into the next case.
- `switch` works on integers and chars, not on strings or doubles.

Classwork Problems

1. Calculator using switch

```
#include <iostream>
using namespace std;

int main() {
    double a, b;
    char op;

    cout << "Enter expression (e.g. 5 + 3): ";
    cin >> a >> op >> b;

    switch (op) {
        case '+':
            cout << "Result: " << (a + b) << endl;
            break;
        case '-':
            cout << "Result: " << (a - b) << endl;
            break;
        case '*':
            cout << "Result: " << (a * b) << endl;
            break;
        case '/':
```

```

        if (b != 0) cout << "Result: " << (a / b) << endl;
        else cout << "Error: division by zero" << endl;
        break;
    default:
        cout << "Invalid operator" << endl;
    }
    return 0;
}

```

2. Day of the week from a number

```

#include <iostream>
using namespace std;

int main() {
    int day;
    cout << "Enter day number (1-7): ";
    cin >> day;

    switch (day) {
        case 1: cout << "Monday" << endl; break;
        case 2: cout << "Tuesday" << endl; break;
        case 3: cout << "Wednesday" << endl; break;
        case 4: cout << "Thursday" << endl; break;
        case 5: cout << "Friday" << endl; break;
        case 6: cout << "Saturday" << endl; break;
        case 7: cout << "Sunday" << endl; break;
        default: cout << "Invalid day number" << endl;
    }
    return 0;
}

```

3. Vowel check using switch

```

#include <iostream>
using namespace std;

int main() {
    char ch;
    cout << "Enter a letter: ";
    cin >> ch;

    switch (ch) {
        case 'a': case 'e': case 'i': case 'o': case 'u':
        case 'A': case 'E': case 'I': case 'O': case 'U':
            cout << ch << " is a vowel" << endl;
            break;
        default:
            cout << ch << " is a consonant" << endl;
    }
    return 0;
}

```

Homework Problems

1. Menu-driven program

```

#include <iostream>
using namespace std;

int main() {
    int choice;
    cout << "1. Add\n2. Subtract\n3. Multiply\n4. Divide\n";
    cout << "Enter your choice: ";
    cin >> choice;

    double a, b;
    cout << "Enter two numbers: ";
    cin >> a >> b;

    switch (choice) {

```

```

        case 1: cout << "Sum: " << (a + b) << endl; break;
        case 2: cout << "Difference: " << (a - b) << endl; break;
        case 3: cout << "Product: " << (a * b) << endl; break;
        case 4:
            if (b != 0) cout << "Quotient: " << (a / b) << endl;
            else cout << "Error: division by zero" << endl;
            break;
        default: cout << "Invalid choice" << endl;
    }
    return 0;
}

```

2. Number of days in a month

```

#include <iostream>
using namespace std;

int main() {
    int month;
    int year = 2024;
    cout << "Enter month number (1-12): ";
    cin >> month;

    int days;
    switch (month) {
        case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            days = 31;
            break;
        case 4: case 6: case 9: case 11:
            days = 30;
            break;
        case 2:
            if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))
                days = 29;
            else
                days = 28;
            break;
        default:
            days = -1;
    }

    if (days == -1) cout << "Invalid month" << endl;
    else cout << "Days in month: " << days << endl;
    return 0;
}

```

Lesson 2: Characters & Strings

Quick recap: char type and ASCII • char arithmetic • std::string: length, indexing, concatenation • helpers

Theory & Nuances

char type and ASCII

A char is stored internally as a small integer — 'A' is literally the number 65 under the hood (standard ASCII runs 0-127). This is exactly why a char can be printed as a letter with cout but also used directly in arithmetic.

char arithmetic

Because chars are just numbers, adding/subtracting integers shifts a letter within the alphabet — 'a' + 1 gives 'b'. The gap between an uppercase letter and its lowercase version is always 32 in ASCII ('a' - 'A' == 32), which is why the classic case-conversion trick is ch +/- 32.

std::string basics

A string is a sequence of characters. .length() (or .size(), identical) gives the number of characters. str[i] accesses the character at position i, using 0-based indexing — the first character is str[0]. + concatenates

two strings into a new one.

helpers

Functions like `tolower()`, `toupper()`, `isdigit()`, `isalpha()`, `isspace()` classify or convert a single character without hand-rolled ASCII arithmetic — they're safer and clearer than tricks like `ch - 32`, especially because they behave correctly even if `ch` isn't a letter.

Key Points / Common Mistakes

- String indices are 0-based: the last character is `str[str.length() - 1]`, not `str[str.length()]`.
- `'a'`, `'A'`, `'0'` in single quotes are just numbers underneath — comparing chars directly works.
- Reading `str[i]` out of bounds doesn't always crash — it can silently read garbage, so always check `i < str.length()`.

Classwork Problems

1. Print the ASCII of a character

```
#include <iostream>
using namespace std;

int main() {
    char ch = 'A';
    int asciiValue = ch; // implicit char -> int

    cout << "Character : " << ch << endl;
    cout << "ASCII value: " << asciiValue << endl;
    return 0;
}
```

2. Convert case using ASCII

```
#include <iostream>
using namespace std;

int main() {
    char ch = 'g';

    char upper = ch - 32; // 'a'-'z' to 'A'-'Z'
    char original = upper + 32;

    cout << "Original : " << ch << endl;
    cout << "Uppercase : " << upper << endl;
    cout << "Back to lower: " << original << endl;
    return 0;
}
```

3. Count vowels in a string

```
#include <iostream>
#include <cctype>
using namespace std;

int main() {
    string text = "Hello World";
    int count = 0;

    for (int i = 0; i < text.length(); i++) {
        char ch = tolower(text[i]);
        if (ch=='a' || ch=='e' || ch=='i' || ch=='o' || ch=='u') {
            count++;
        }
    }

    cout << "Vowel count in \"" << text << "\": " << count << endl;
    return 0;
}
```

Homework Problems

1. Reverse a string

```
#include <iostream>
using namespace std;

int main() {
    string text = "hello";
    string reversed = "";

    for (int i = text.length() - 1; i >= 0; i--) {
        reversed += text[i];
    }

    cout << "Original: " << text << endl;
    cout << "Reversed: " << reversed << endl;
    return 0;
}
```

2. Check a one-word palindrome

```
#include <iostream>
using namespace std;

int main() {
    string word = "madam";
    bool isPalindrome = true;

    int n = word.length();
    for (int i = 0; i < n / 2; i++) {
        if (word[i] != word[n - 1 - i]) {
            isPalindrome = false;
            break;
        }
    }

    cout << boolalpha;
    cout << "\"" << word << "\" is a palindrome: " << isPalindrome << endl;
    return 0;
}
```
